

Evaluación 1 – Programación 3

Objetivo

El objetivo de esta evaluación es consolidar los conocimientos adquiridos en las primeras unidades del cursado (HTML, CSS, JavaScript y TypeScript), mediante la evolución del proyecto Food Store hacia una aplicación frontend más dinámica e interactiva.

Consigna

A partir del proyecto desarrollado en los Trabajos Prácticos de:

- HTML
- CSS
- JavaScript
- TypeScript (Trabajo Práctico Integrador)

deberás extender la aplicación actual incorporando las siguientes funcionalidades.

Requerimientos obligatorios

1. Carrito básico con persistencia

Implementar un carrito de compras utilizando localStorage que permita:

- Agregar productos desde el catálogo
- Visualizar los productos agregados en una vista de carrito
- Mostrar nombre, precio y cantidad de cada producto
- Calcular y mostrar el total de la compra

No es necesario implementar checkout ni conexión con backend.

2. Búsqueda y filtrado de productos

Incorporar funcionalidades de interacción sobre el catálogo:

- Búsqueda de productos por nombre
- Filtrado por categoría (desde el menú lateral o equivalente)

No se requiere implementar ordenamientos ni combinaciones avanzadas de filtros.

Requerimiento técnico: configuración del entorno

Punto de partida

Podés resolver el parcial de cualquiera de estas dos formas:

- Continuando tu propio proyecto del **Trabajo Práctico Integrador de TypeScript** (recomendado): ya tenés la estructura de carpetas y la configuración de Vite armadas.
- **Clonando nuevamente el repositorio base** provisto por la cátedra, que ya incluye Vite + TypeScript configurados.

En cualquiera de los dos casos, la estructura final del proyecto debe coincidir con la que se detalla más abajo.

```
# 1. Clonar el repositorio base (si no continuás tu proyecto del TP)
git clone https://github.com/chiro45/proteger_rutas
cd proteger_rutas

# 2. Instalar dependencias con pnpm
pnpm install

# 3. Levantar el servidor de desarrollo
pnpm dev
```

El servidor de desarrollo estará disponible en <http://localhost:5173>.

El gestor de paquetes del proyecto es pnpm (ya configurado en el repo): usalo para instalar y para levantar el servidor.

Si todavía no lo tenés, se habilita con `corepack enable pnpm` (viene incluido con Node.js).

Estructura esperada

Las páginas y la lógica del parcial deben ubicarse dentro de `src/` respetando la organización que ya venís usando desde el Trabajo Práctico Integrador de TypeScript:

```
src/
├── pages/
│   ├── auth/           ← ya existente (registro y login)
│   ├── admin/          ← ya existente
│   └── client/         ← vistas del cliente
│       ├── home/
│       │   ├── home.html ← catálogo de productos
│       │   └── home.ts   ← render, búsqueda y filtros
│       └── cart/
│           ├── cart.html ← vista del carrito
│           └── cart.ts   ← render del carrito y total
├── types/
│   ├── product.ts      ← interfaces IProduct e ICartItem
│   └── categoria.ts    ← interface ICategoria
├── data/
│   └── data.ts         ← PRODUCTS y getCategories()
├── utils/
│   └── cart.ts         ← lógica del carrito (localStorage)
```

Las carpetas `pages/`, `types/` y `utils/` ya existen en el proyecto del TP: en este parcial solo agregás las vistas `client/home/` y `client/cart/`, la carpeta `data/` y el archivo `utils/cart.ts`.

Convenciones del proyecto

- **Interfaces:** todas las interfaces se nombran con el prefijo `I`, igual que `IUser` del TP integrador: `IProduct`, `ICartItem`, `ICategoria`.

- **Claves de localStorage:** el proyecto usa "users" y "userData" (definidas en el TP) y suma "cart" para el carrito de este parcial.
- **Lógica reusable:** las funciones del carrito (agregar, actualizar cantidad, obtener ítems, calcular total) van en `src/utls/cart.ts`; las vistas solo renderizan y escuchan eventos.
- **Datos:** los productos y categorías se centralizan en `src/data/data.ts` (constante `PRODUCTS` y función `getCategories()`), tipados con las interfaces de `types/`.

Registrar las páginas en `vite.config.ts`

Vite necesita conocer todos los archivos HTML del proyecto para incluirlos en el build. Debés agregar tus páginas nuevas en la sección `build.rollupOptions.input` de `vite.config.ts`:

```
import { defineConfig } from 'vite';
import { resolve } from 'path';

export default defineConfig({
  build: {
    rollupOptions: {
      input: {
        // páginas existentes del proyecto (TP integrador):
        index: resolve(__dirname, 'index.html'),
        authRegistro: resolve(__dirname, 'src/pages/auth/registro/registro.html'),
        authLogin: resolve(__dirname, 'src/pages/auth/login/login.html'),

        // agregá tus páginas del parcial:
        clientHome: resolve(__dirname, 'src/pages/client/home/home.html'),
        clientCart: resolve(__dirname, 'src/pages/client/cart/cart.html'),
      },
    },
  },
  base: './',
});
```

Si no registrás una página en `vite.config.ts`, no será incluida en el build y no podrá navegarse correctamente al ejecutar `pnpm build`. Esto aplica también a las páginas que creaste en el TP integrador (registro y login): revisá que estén todas listadas.

Consideraciones técnicas

- El desarrollo debe realizarse exclusivamente con HTML5, CSS3, JavaScript y TypeScript
- No se permite el uso de frameworks (React, Angular, etc.)
- Los datos de productos y categorías se cargan desde `src/data/data.ts`; el carrito se persiste en `localStorage`
- Se debe respetar la estructura de carpetas y las convenciones de nombres detalladas más arriba

En esta instancia no se evalúa la protección de rutas ni la autenticación.

Si tu proyecto viene del TP integrador y ya tiene el guard implementado, dejalo tal como está: no hace falta agregar, quitar ni modificar esa lógica para el parcial.

Entrega

La entrega deberá incluir:

1. Repositorio

- El código debe estar en un único .zip con un .txt con el link del video
- El proyecto debe ser funcional y ejecutable
- Debe incluir un archivo README.md con:
 - Descripción breve del proyecto
 - Instrucciones para ejecutarlo

2. Video de presentación (obligatorio)

Se deberá entregar un video con las siguientes características:

- Duración: entre 10 y 15 minutos
- Cámara encendida durante toda la exposición
- Audio claro y comprensible

Contenido del video

En el video deberás:

- Presentarte brevemente
- Mostrar el funcionamiento de la aplicación: catálogo de productos, carrito, búsqueda y filtros
- Explicar qué funcionalidades lograste implementar, cómo abordaste la resolución y qué dificultades encontraste (si las hubo)

Criterios de evaluación

Se tendrá en cuenta:

- Correcto funcionamiento de las funcionalidades solicitadas
- Uso adecuado de JavaScript y TypeScript (interfaces y tipos, sin abuso de any)
- Manipulación del DOM
- Uso de localStorage
- Organización del código: respeto de la estructura de carpetas y de las convenciones de nombres
- Registro correcto de las páginas en vite.config.ts (el proyecto compila y navega)
- Claridad en la explicación del video
- Presentación general del proyecto

Importante

No se evaluará la conexión con backend ni la protección de rutas en esta instancia.

El foco está puesto en la lógica frontend y la interacción con el usuario.

Historias de Usuario de referencia para esta Evaluación

HU-P1-01: Buscar productos en el catálogo

Como cliente de Food Store,
quiero poder buscar productos por nombre dentro del catálogo,
para encontrar más rápido el producto que deseo visualizar.

Criterios de aceptación:

- Debe existir un campo de búsqueda visible en la pantalla principal del catálogo.
- Al escribir un texto, se deben mostrar únicamente los productos cuyo nombre coincida total o parcialmente con la búsqueda.
- Si no hay coincidencias, se debe informar visualmente al usuario.
- La búsqueda debe realizarse sobre los productos cargados dinámicamente.

Dónde implementarlo:

- Vista del catálogo: `src/pages/client/home/home.html`.
 - Lógica de render y filtrado en `src/pages/client/home/home.ts`, sobre los datos de `src/data/data.ts`.
-

HU-P1-02: Filtrar productos por categoría

Como cliente de Food Store,
quiero poder filtrar los productos por categoría,
para visualizar únicamente aquellos que pertenecen al tipo de comida que me interesa.

Criterios de aceptación:

- Las categorías deben mostrarse en el menú lateral o sección equivalente.
- Al seleccionar una categoría, deben mostrarse solo los productos de esa categoría.
- Debe poder volver a visualizarse el catálogo completo.
- El filtrado debe aplicarse sobre los datos ya cargados en memoria.

Dónde implementarlo:

- Listado de categorías obtenido desde `getCategories()` en `src/data/data.ts`, tipado con `ICategoria`.
 - Misma lógica de renderizado dinámico de `home.ts` utilizada para cargar productos.
-

HU-P1-03: Agregar productos al carrito

Como cliente de Food Store,
quiero poder agregar productos al carrito desde el catálogo,
para ir armando mi compra.

Criterios de aceptación:

- Cada producto debe tener una acción visible para agregarlo al carrito.

- Al agregar un producto, este debe guardarse en `localStorage` bajo la clave "cart".
- Si el producto ya fue agregado previamente, debe actualizarse su cantidad en lugar de duplicarse como ítem separado.
- Debe existir algún indicador visual de que la acción se realizó correctamente.

Dónde implementarlo:

- Botón "Agregar" dentro de cada tarjeta de producto renderizada dinámicamente.
- Lógica del carrito en `src/utls/cart.ts`, usando la interface `ICartItem` definida en `src/types/product.ts`.

HU-P1-04: Visualizar el carrito

Como cliente de Food Store,
quiero visualizar los productos que fui agregando al carrito,
para revisar mi selección antes de una futura compra.

Criterios de aceptación:

- Debe existir una vista o página de carrito accesible desde la navegación.
- En ella deben mostrarse, como mínimo: nombre del producto, precio y cantidad.
- Si el carrito está vacío, debe mostrarse un mensaje indicándolo.
- La información debe recuperarse desde `localStorage` (clave "cart").

Dónde implementarlo:

- Página `src/pages/client/cart/cart.html` con su lógica en `cart.ts`.
- Navegación desde el catálogo hacia el carrito (recordá registrar ambas páginas en `vite.config.ts`).

HU-P1-05: Calcular el total del carrito

Como cliente de Food Store,
quiero ver el total acumulado de los productos cargados en el carrito,
para conocer el importe total de mi selección.

Criterios de aceptación:

- En la vista del carrito debe mostrarse el total general.
- El total debe calcularse como la suma de todos los subtotales de los productos agregados.
- El valor debe actualizarse correctamente según el contenido almacenado.

Dónde implementarlo:

- Vista del carrito (`cart.html` / `cart.ts`).
- Función de cálculo dentro de la lógica del carrito, en `src/utls/cart.ts`.

Anexo: resumen de archivos y responsabilidades

Cuadro de referencia rápida para chequear la entrega antes de comprimir el .zip.

Archivo / carpeta	Responsabilidad
src/pages/client/home/home.html	Marcado del catálogo: campo de búsqueda, menú de categorías y contenedor de productos.
src/pages/client/home/home.ts	Render dinámico de productos y categorías, búsqueda por nombre y filtrado por categoría.
src/pages/client/cart/cart.html	Marcado de la vista del carrito y del total.
src/pages/client/cart/cart.ts	Render de los ítems del carrito, cantidades y total.
src/types/product.ts	Interfaces IProduct e ICartItem.
src/types/categoria.ts	Interface ICategoria.
src/data/data.ts	Constante PRODUCTS y función getCategories().
src/utls/cart.ts	Lógica del carrito sobre localStorage (clave "cart"): agregar, actualizar cantidad, leer ítems y calcular total.
vite.config.ts	Registro de home.html y cart.html en build.rollupOptions.input.
README.md	Descripción breve del proyecto e instrucciones de ejecución.